Measuring a measurement: self-generated labels overestimate probe performance on reasoning robustness

Anonymous Author(s)
Submission under anonymous review

Abstract

We pre-registered and ran more than twenty hypotheses for predicting whether a model's correct answer to an olympiad problem is *robust* (survives answer-preserving perturbation) or *spurious*. Almost none cleared its null, and the one rule that did is unaffordable at test time. The useful result is why the evaluation is uninformative. We generated 93 labels at $k{=}8$ samples; the organisers' labels resolve `max_drop` to 0.01, implying ≈ 100. On our labels a mid-generation hidden-state probe reaches AUC 0.872; the same features under the same protocol reach 0.573–0.610 on theirs, by transfer and by direct training. We rule out the classifier (five variants, learned parameters $24 \to 0$) and the label rule (our thresholds reproduce theirs 82/82). At $n{=}33$ the shuffle-null band is ≈ 0.41 wide and nothing clears it on accuracy, problem length included. We add three cheap checkpoints that each caught a self-deception: a length control that unmasked our first probe, a stability test showing $\approx 1/4$ of our labels flip on re-run, and a three-parameter fit whose below-chance score was nearly read as one feature's direction. Report the resolution and re-run stability of self-generated labels next to any probe score obtained on them.

1 Introduction

The task gives a method a `model_id` and a problem statement and asks for one bit: will this model's correct answer survive answer-preserving perturbations? A label of *spurious* marks a problem the model answers correctly but loses under edits that preserve the answer, which makes the target a property of the model–problem pair rather than of the text — and is why the organisers' own baselines read internal states (probing, $58.37\% \pm 7.5$) and sampling behaviour (CoT uncertainty, $69.23\% \pm 10.13$) rather than surface features [1].

Our contribution is not a better detector. We ran the obvious white-box programme — linear probes on hidden states before and during generation, self-consistency and memorisation signals, causal knockouts, test-time perturbation — under a fixed pre-registration discipline, and report it as a negative result with a measured cause.

Claims. (1) **Self-label resolution bounds what a probe score means**: our $k{=}8$ labels resolve `max_drop` to 0.125, the organisers' to 0.01; the same probe scores 0.872 on ours and 0.573–0.610 on theirs (§3). (2) **The gap is neither the classifier nor the label rule** (§3). (3) **Three cheap checkpoints are what kept the programme honest** (§4). (4) **At $n{=}33$ nothing beats a constant predictor on accuracy, length included** (§5); the one exception needs a token budget the evaluation does not grant (§6).

2 Setup

Labels. Public training set `aimo-interp/train-main-v2`: 82 rows, 36 problems, 5 models; 71 rows carry a label (robust 52 / spurious 19) and 11 are withheld. Aggregated to problem level by majority this gives **33 labels (robust 20 / spurious 13)**, so a constant "robust" predictor scores 0.606. Labels are problem-determined: all 11 problems appearing under two or more models carry the same label across models, so a model$\times$problem interaction is absent by construction. We also generated 93 labels of our own (3 models, ≈ 30 problems each) from MATH level 5 and AIME with answer-preserving perturbations.

Table 1: The same features and protocol, two label sources. Window-2048 mid-generation mean-pooled hidden state at layer $L/2$, PCA-24, ℓ_2 logistic, class-balanced, inner-CV C, problem-level folds, 1000-permutation null.

Label source	`max_drop` grid	Skywork	Olmo	DeepSeek	pooled / mean
Ours, k=8 (93 rows)	0.125	0.981	0.762	0.874	**0.872**
Organisers, transfer	0.01	0.642	0.627	0.562	**0.610**
Organisers, direct training	0.01	0.565	0.638	0.515	**0.573**

Budget. One RTX PRO 6000 (96 GB), **3600 s for the entire prediction run** (official FAQ 8; the README adds "not one hour per problem"). This single constraint decided most of this report: the signals we found live at generation caps of 8192 tokens and above, which a final-set run cannot buy, and faithful replication of the label definition needs 24576 — 187% of the budget even at the smallest perturbation count. [`0fe94a2`: `ttp_budget.txt`, `h22_leaderboard_budget.txt`]

Pre-registration protocol. Every hypothesis was fixed before measurement in a versioned notebook: primary metric, sign, null procedure (problem-level label shuffling, 1000 permutations, the full pipeline re-run inside each), a length-residual companion, and the pass criterion. One evaluation run per pre-registration; signs were never flipped after the fact, and where a result came out opposite to the registered direction we report it opposite. Anything decided after seeing numbers is labelled exploratory here and in the code.

3 Self-label resolution, and what it does to a probe score

The resolution gap is measured, not inferred. The organisers' `max_drop` takes 35 distinct values on a 0.01 grid over $[-0.090, +1.000]$ — six rows are negative, i.e. the perturbed problem was solved *more* often — which implies on the order of 100 samples per condition. Our k=8 grid is 0.125: two failures out of eight in one perturbation type is the smallest event that can cross the spurious threshold, against roughly 25 of 100 for them. [`8adae75`: `org_label_rule.txt`]

The probe score does not survive the label change. Transfer freezes the weights trained on our labels and applies them to the organisers' 33 problems; direct training re-fits on their labels under leave-one-problem-out. Both land at 0.573–0.610, inside their shuffle nulls (upper bounds 0.71 and 0.72), against 0.872 on our own. The problem sets do not overlap: exact-match 0 and maximum 8-gram Jaccard 0.000, because our pool generator excludes the official problems. [`612707b`: `org_sub2048_transfer.txt`], [`a1e7498`: `org_train_report.txt`]

It is neither the classifier nor the label rule. Five classifiers on identical rows, folds and null draws span only 0.109 (PCA-24 + logistic 0.872 down to per-fold univariate selection 0.640), and reducing learned parameters makes it *worse*, not better; re-drawing rows from the same design gives a 95% interval of width 0.49–0.76, wider than any design difference. Our reverse-engineered thresholds reproduce the organisers' `is_robust` **82/82** on all public rows, so the rule is right; and perturbation strength is capped by answer preservation — per-type collapse probability 10.9% against the 40–50% their 27% spurious rate requires, with *more* perturbation types making a majority *less* likely. Details, controls and commits in Appendix 7.

4 Three checkpoints that caught us

Each cost one script and no GPU time, and each reversed a conclusion we had already written down.

(a) A length control, because the first probe was reading problem length. Our first probe scored cross-validated balanced accuracy 0.634 on 137 problems and we believed it. Word count alone scored AUC 0.75 against the probe's 0.72; the probe's margin correlated with length at $\rho = +0.71$; adding the probe to length moved nothing ($0.741 \rightarrow 0.740$); and restricting to problems of at least 15 words dropped the probe to 0.44, at least 20 words to 0.36. The cause was in the labels, not the model: one perturbation type (Domain) destroyed 50 of 52 problems of twelve words or

fewer while the other four types were length-independent, so that label set largely recorded a domain rewrite breaking a short problem. [64c6ca3: length_confound.py, topic_confound.py]

Residualising a confound is the usual response and it is weaker than it looks, because it also removes real signal and the reader cannot tell which happened. The stronger test is a second data set where the confound's sign is *reversed*, and we had one: on the official 33 problems shorter problems are robust (length→robust AUC 0.669), on our own 88 longer ones are (0.372). Memorisation scores followed the flip to within 0.017–0.035 AUC (0.704 then 0.389), identifying them as a length proxy in a way residualisation cannot. That closed five hypotheses at once. [64c6ca3: h23_length_proxy.txt]

(b) A label-stability test, because about a quarter of our labels flip on re-run. Scores on small label sets move because the labels move. Re-labelling 20 problems with only the decoding temperature changed, $0.6 \rightarrow 1.0$ with top-k 40, flipped **7 of 20 labels** and pushed one into the excluded band: agreement 65%. Independently, removing a detokenisation defect and re-sampling the same two DeepSeek slices gave 62% and 67% agreement. Two different interventions produced the same magnitude of change, which is what resampling variance looks like rather than an effect of either intervention. A third measurement isolates it: two of our runs happened to overlap on the same problems at the same settings, and on those the pure re-sample agreement was 94% at k=8 versus 74% at k=4. [261131c: temp_sens_report.txt], [0b72579: dsrelabel_compare.txt], [4fcd838: flipent_report.txt]

The consequence is quantitative. A method evaluated on 30 labels of which a quarter are unstable cannot be compared to another method at a resolution finer than that instability; our own k=4-versus-k=8 comparison on the *same* 30 problems moved one score from 0.721 to 0.571 with nothing changed but the label rule's sample count. Reporting re-run agreement next to a self-label score costs one extra run and tells the reader how much of the score is addressable.

(c) Do not read a multivariate fit's score as a single feature's direction. We had observed, five times under four different instruments, that *robust* problems move *more* under answer-preserving edits than the pre-registered direction expected. One of those five was a three-feature logistic fit on entropy-trajectory features that produced a leave-one-problem-out AUC of $\mathbf{0.123}$ for one model — far below chance — and we were one step from treating 0.123 as strong evidence about the entropy feature's sign. It is not: in the same run the three features *individually* scored 0.531, 0.515 and 0.550, i.e. nothing. The 0.123 is what a three-parameter fit does to 33 rows when it latches onto the wrong direction, and the strongest regularisation in the grid was selected in 27 of 33 folds without preventing it.

We then pre-registered and ran the flip as a *single*-feature test on four self-labelled arms it had never been evaluated on. All four stayed inside their shuffle nulls (0.721, 0.562, 0.337, 0.526 against upper bounds 0.740, 0.686, 0.756, 0.618), and the model that had motivated the flip came out at 0.337 — the *original* direction, not the flipped one. The flip is not supported, and the apparent evidence for it was an artefact of reading a fit as a feature. [0fe94a2: entlens_report.txt], [4fcd838: flipent_report.txt]

Before accepting any of these reversals we audited the labels themselves: five checks — label dtype, both sign conventions, text alignment, $\rho(\texttt{is_robust}, \texttt{max_drop})$ and an end-to-end planted label — all passed, so the reversals are not bookkeeping (Appendix 7). [e0feda3: label_audit.txt]

5 At n=33, nothing beats a constant predictor

Table 2 is the whole programme on one axis, and three things read off it. *First, the constant predictor is hard to beat and most of our work does not beat it* — fifteen of twenty rows sit below 0.606, and the AUC column agrees, with seven rows below 0.5. *Second, problem length is the honest baseline and it is not significant either*: AUC 0.671, the highest of any non-generation method, yet inside its own shuffle null (upper bound 0.70), with accuracy 0.636 one problem above the constant. Our submitted length rule therefore rests on calibration rather than ranking — score $-\log(\text{words})$, threshold fixed so the predicted True rate equals the label prior ($20/33 = 0.6061$; threshold -4.197174, achieved rate exactly 0.6061) — and it loads no model and generates no tokens, 9.15 ms for an 8-model, 40-problem run, so the failure modes that voided our generation-based submissions cannot occur. [cb9c291: lencal_artifacts.txt] *Third, the inner cross-validation keeps saying the same thing*: it picked the strongest regularisation in the grid in 29 of 33 folds for one model and 22 of 33 for another, and switching from our labels to the organisers' did not change that, which locates the symptom in the dimension-to-row ratio rather than in label noise. Appendix B makes it concrete: with a one-dimensional signal planted at known strength into the real evaluation path, our pipeline recovers only 0.696

Table 2: Every method we evaluated on the organisers' 33 problem-level labels, against the two baselines. Accuracy is the decision rule's accuracy with thresholds fixed inside training folds only; "clears" is accuracy above that method's own 1000-permutation shuffle-null 95% upper bound. H-FIN rows are exploratory (§6); R3 uses a different 19-problem label set and is not directly comparable. Full per-hypothesis list in Appendix A.

	method	accuracy	AUC	clears own null
baseline	constant "all robust"	**0.606**	0.500	—
baseline	problem length $-\log(\text{words})$, LOO threshold	**0.636**	0.671	no
probes on internal states				
P1	problem-text probe, model-mean (pre-registered primary)	0.545	0.616	no
P2	mid-generation probe, window 2048, direct training	0.556	0.573	no
P3	mid-generation probe, transfer with frozen weights	0.505	0.610	no
P4	+ in-call rank 50% cut instead of fixed threshold	0.545	0.610	no
P5	+ 1:1 rank average with length	0.626	0.674	no
P5a	+ *ablation*: one model's probe scores every problem	0.485–0.545	0.562–0.642	no
P5b	+ *ablation*: per-model thresholds replaced by one	0.495	0.610	no
P6	mass-mean direction, transfer	0.586	0.633	no
P7	per-fold univariate selection (exploratory)	0.656	0.631	no
behaviour read from short generations				
B1	entropy trajectory, 3 features, LOPO	0.424	0.413	no
B2	logit-lens layer of answer commitment	0.606	0.540	no
B3	forced-answer agreement under perturbation, 512-tok CoT	0.530	0.455	no
B4	accuracy drop vs. pseudo-gold, 1024-tok CoT, 3 samples	0.535	0.481	no
B5	self-disagreement of 3 original samples	0.475	0.405	no
B6	accuracy direction fitted on 156 held-out rows, projected once	0.545	0.590	no
B7	accuracy drop vs. **gold** (19-problem set, both signs)	0.579	0.526	no
replicating the label definition at test time				
D1	N=4, K=6 answer-preserving perturbations, cap 24576	0.645	0.673	no
natural termination (exploratory, and see §6)				
H1	cap 12288, DeepSeek	0.485	0.575	no
H2	cap 12288, Skywork	0.697	0.683	at bound
H3	cap 12288, Olmo	**0.818**	0.850	✓
H4	3-model mean	0.667	—	✓
H5	per-model routing hybrid (H2, H3 + length elsewhere)	**0.682**	—	✓

from a 20σ signal at the dimensionality we use. A probe scoring 0.573 here is inside the range where a planted signal would also have scored. [`e0feda3`: `label_audit.txt`]

6 Acting instead of probing: the budget wall, measured against gold

If a probe cannot read the label off internal states, the remaining option is to *act*: load the judged model, perturb the problem in ways that preserve the answer, and watch the answer. We built that as a submission and ran five pre-registered rounds (B3–B7 and D1 in Table 2).

The mechanism works; the budget kills it. Cutting the chain of thought at a cap and then *forcing* an answer makes every sequence emit an integer (94.2–100% extraction across four models) for 911 s, 28% of the budget. What the budget cannot buy is *being right first*. On 19 of our own problems where gold answers exist, the original answer at an affordable cap is almost never correct — accuracy $\mathbf{0.175}$, 14 of 19 at exactly zero — while the labels for those same problems were made at a 32768-token cap with eight samples, where accuracy is $\mathbf{0.868}$. The gap is $\mathbf{0.693}$, and since the statistic is base_acc $-\min_t$ var_acc$_t$, a base accuracy of zero leaves nothing to drop: the score took one distinct value across the set, so both pre-registered signs returned the identical number. The metric did not point the wrong way; it ceased to exist. [`f79c33f`: `ttp_r3_report.txt`]

And the substitute everyone uses is quantitatively wrong. Lacking gold answers, every earlier test-time replication — ours included — scored "did the answer change" instead of "did the model get it wrong". On those same 19 problems the substitute reports 0.842 where gold reports 0.175: a $\mathbf{4.8}\times$ overestimate, because a consistently wrong model has a stable majority answer. What a submission can buy is the stability of an error; what the label asks for is the fragility of a correct solution. Implementing the definition faithfully (N=4, K=6 answer-preserving perturbations, majority rule, cap 24576) reaches AUC 0.673 inside its null, and a reduced K=2 rename-only version of itself reaches 0.675 — perturbation diversity moved the AUC by 0.002, so faithfulness was not the missing piece (Appendix 7). [`0fe94a2`: `h22_official33_skywork_report.txt`]

One rule does clear its null — and cannot be bought. The exception is the simplest behavioural signal in the set and not ours originally: a public competitor repository reports a non-probing rule in which a problem is robust if its attempts terminate naturally inside the token cap [2]. We had measured that quantity as a continuous completion rate (AUC 0.671 at cap 16384, 0.779 at cap 8192) but never as a decision rule, having closed the generation-length family as budget-bound. Re-evaluated as an absolute rule — robust iff at least half the attempts finish inside the cap — on stored generations at cap 12288 it gives accuracy 0.818 for Olmo (AUC 0.850, null upper 0.66), 0.697 for Skywork and 0.485 for DeepSeek: a three-model mean of $\mathbf{0.667}$. Routing the two models that clear their nulls to this rule and the other two to the length rule gives $\mathbf{0.682}$ against a null upper bound of 0.637 — the highest number we measured on these labels and the only one above both baselines. [98d20b7: `hfin_report.txt`, `hfin_hybrid_report.txt`]

Three caveats make this a measurement rather than a method. **(i) The routing is post-hoc** — the two models were chosen *after* seeing per-model accuracies, so the pre-registrable object is the three-model mean 0.667. **(ii) The setting that works cannot be bought at final-set scale**: only cap 12288 clears the gate (cap 8192 gives 0.546, cap 4096 0.404), and under our measured timing model 35 problems at that cap cost 2802 s for Skywork and 3650 s for Olmo against a 3300 s total, so at final-set scale both models fall back and the submission *becomes* the length rule. Lowering the cap does not help: "finished naturally" saturates to False, which is silence, not signal. **(iii) The rule may be re-reading the label definition** — the organisers' labels are accuracies inside a fixed budget, so "does it finish inside a budget" is partly the same measurement, which breaks transfer whenever that budget differs.

7 Conclusion

Three numbers carry this report. A probe scoring 0.872 on labels we generated scored 0.610 when those weights were applied to the organisers' labels and 0.573 when refitted on them, and the difference is traceable to resolution: our labels resolve accuracy to $1/8$, theirs to $1/100$, and about a quarter of ours flip on re-run. The definition can be reproduced inside a submission but not usefully, because the accuracy it needs is 0.868 at the budget the labels were made with and 0.175 at the budget a submission can buy. And at $n{=}33$ our own pipeline recovers only 0.696 from a 20σ planted signal, which bounds how strongly any null here may be read, ours included. So: when reporting a probe score obtained on labels you generated, report the resolution of those labels and their agreement on an independent re-run beside it, and treat the score as an upper bound until it survives a label source you did not generate.

Limitations

We cannot separate "the feature does not carry this label" from "$n{=}33$ cannot detect it"; that separation needs more labels, which is exactly what the resolution argument says is expensive. Our own labels cover three of the five judged models and ≈ 30 problems each, and they are filtered: label generation kept only problems with `base_acc` ≥ 0.5, and on that subset the accuracy axis is nearly constant (standard deviations 0.120, 0.167, 0.073; for one model every greedy continuation was correct, $32/32$). Including the problems the screening stage rejected raises those to 0.215, 0.205, 0.284. Any claim of the form "the probe reads whether the model knows this problem" has to contend with a label set built by discarding the problems it does not know. [73ca8d1: `r5_rows.txt`]

The mid-generation features use one greedy continuation per problem while the labels aggregate roughly 100 stochastic ones; we tested a 4-sample average at the labels' own decoding settings and it was *worse* than greedy (0.757 vs 0.872), so the asymmetry is unresolved rather than dismissed. [58b2e7d: `probe_gen_report_samp.txt`] The model$\times$problem component is unmeasurable on the public training set by construction, since all 11 problems appearing for more than one model carry identical labels. The H-FIN result in §6 is computed from stored generations rather than a fresh pre-registered run, its routing is post-hoc, and we have not measured it on any label source other than the organisers' 33. Two measurement defects we found late are reported in Appendix C; both inflated or deflated a model's apparent capability and both were invisible in the aggregates we had been storing.

Why we do not lean on leaderboard position. The small track has 14 binary items and allows 100 submissions, so a participant can recover the entire label vector by flipping one item at a time in about 15 submissions. At the time of writing the public board holds 1726 submissions, with 9 perfect scores and 53 above 0.86 on the small track and 1 perfect on the main track; medians are 0.57 and 0.52. We did not probe the labels and we do not report rank. Every number in this report is measured against the public training labels or our own, never against the hidden set. [98d20b7: `codabench_ids.txt`]

Finally, a reproducibility limitation of our own release: the development repository tracks 3.5 GB of feature caches including six files above GitHub's 100 MB limit, so the public release is a snapshot containing all code and all measurement logs (664 files, 13 MB) but not the `.npz` caches. Commit hashes cited above refer to the private development history.

AI-usage statement

This work was carried out by the author in a continuous collaboration with Claude (Anthropic), used as a coding and analysis agent inside a terminal agent harness. Concretely, the AI wrote essentially all of the code in the released repository — feature extraction, the evaluation scripts, the label-generation pipelines, the submission implementations and the cloud job runners — executed the measurements, maintained the pre-registration notebook, and drafted this report including its tables. The author set the research direction, chose and approved every hypothesis and pre-registration before measurement, decided the budget and which experiments to buy, performed all Codabench uploads, and made the final calls on what to claim. Verification was treated as the author's responsibility and as an explicit standing rule for the agent: every number in this report is printed by a committed script into a committed log, several of the corrections and retractions recorded here originated in the author challenging an AI-stated conclusion, and the three checkpoints in §4 were instituted after specific AI-authored misreadings were caught. No AI system is an author; responsibility for the content is the author's.

References

[1] AIMO Interpretability Challenge organisers. *Detecting spurious reasoning in mathematical problem solving*. arXiv:2607.13899, 2026. (Task definition, label definition, and the probing and CoT-uncertainty baselines we compare against.)

[2] ECONLEARN. *aimo-interp-difficulty-vs-fragility*. Public repository, 2026. `https://github.com/EconLearn/aimo-interp-difficulty-vs-fragility` (Non-probing natural-termination rule; reports leave-one-template-out ≈ 0.845 on the 82 public rows and 17/21 on the main track. We reached the same quantity independently from our own completion-rate measurement and did not use their code.)

[3] BYEOLKI. *aimo-interp*. Public repository, 2026. `https://github.com/byeolki/aimo-interp` (Reports that a prompt-only probe reads solve rate rather than robustness. Our §5 row B6 tests that reading directly and finds the solve-rate direction is not even recovered in-domain, with cross-validated correlations 0.102, -0.067, 0.172.)

Supporting detail (moved from the body for the page limit)

Verbatim measurement detail for the compressed claims in §3, §4 and §6.

It is not the classifier. If over-fitting at 30 rows were the bottleneck, reducing learned parameters should help; it does the opposite. On identical rows, folds and null draws: PCA-24 + logistic 0.872, mass-mean (one direction) 0.868, a hyperdimensional prototype with no fitted parameters 0.824, the same with 16-segment order encoding 0.763, per-fold univariate selection 0.640. Two controls: ridge on PCA-24 and ridge on the raw $3584/4096$-dimensional features give *identical* scores (max abs. difference 6.0×10^{-8}), because at 30 rows the feature matrix has rank ≤ 29; and the continuous `max_drop` target instead of the binary label lowers the score by 0.088. The span of those five cells is 0.109, whereas re-drawing rows from the same design, labels and evaluation path gives a 95% interval of width 0.49–0.76 (200 draws per model) — so we do not even claim the ordering. [`9c7a10e`: `cmp4_report.txt`], [`f730878`: `probe_subset_shift.txt`]

It is not the label rule either. We had reverse-engineered the organisers' thresholds as `max_drop` $\leq 0.10 \Rightarrow$ robust, $\geq 0.25 \Rightarrow$ spurious, in between excluded. On all 82 public rows that rule reproduces `is_robust` **82/82**: the 11 withheld rows have `max_drop` in $[0.110, 0.210]$, exactly the gap, and a grid sweep pins the lower threshold at 0.10 with only the upper loose. This is *in-sample* — the rule came from these rows — so it locates the thresholds,

it does not validate them. The defect in our labels is resolution and reproducibility, not the decision rule. [8adae75: org_label_rule.txt]

A cheaper symptom of the same problem. Perturbation strength is bounded by answer preservation: across our $k{=}8$ label sets the per-type collapse probability is 10.9%, while the organisers' spurious rate of 27% (19/71) under the same majority rule needs 40–50%. Since the majority threshold rises with the type count, adding types *lowers* the chance a majority fires (3.3% at $T{=}3$, 0.2% at $T{=}6$), and a model-rewritten Domain perturbation passed its checks for only $9/20$ problems against a pre-registered 60% gate. *The perturbations we can build under answer preservation do not make the organisers' rule fire.* [6e76312: strong_need.txt]

We audited the labels before accepting any reversal. Five checks all passed — no string values in the label column (so the classic `bool('False')` inversion cannot occur), both sign conventions verified on hand-built scores, $33/33$ text alignment (which also exposed that only $18/33$ problem indices coincide with label indices — a real misalignment we had hit and fixed), $\rho(\texttt{is_robust}, \texttt{max_drop}) = -0.773$, and a planted label recovered through the real nested-CV code. [e0feda3: label_audit.txt]

The mechanism works; the measurement does not. The design cuts the chain of thought at a cap and then *forces* an answer, so every sequence emits an integer however far the reasoning got: integer extraction was 94.2–100% across four models and three cut points, and a four-model, forty-problem run costs 911 s — 28% of the budget, against the 187% the same design needs at a 2048-token cap. [0fe94a2: ttp_report.txt]

Why it fails: the budget sets the accuracy, and the accuracy sets the metric. Round B7 ran on 19 of our own problems for which gold answers exist, so for once the definition could be evaluated without substituting self-consistency for correctness. At the cap a submission can afford the model's original answer is almost never right: accuracy $\mathbf{0.175}$, median 0.000, 14 of 19 problems at exactly zero — while the same problems' labels were generated at a 32768-token cap with eight samples, where accuracy is $\mathbf{0.868}$. The gap is $\mathbf{0.693}$. Since the statistic is base_acc $-\min_t$ var_acc$_t$, a base accuracy of zero leaves nothing to drop: the score took one distinct value across the set, which is why both pre-registered signs produced the identical number. The metric did not point the wrong way; it ceased to exist. [f79c33f: ttp_r3_report.txt]

And the substitute everyone uses is quantitatively wrong. Every earlier test-time replication of the definition, ours included, replaced "did the model get it wrong" with "did the answer change", because the public training set has no answer column. On the same 19 problems that substitute reports 0.842 where gold reports 0.175: a $\mathbf{4.8}\times$ overestimate. The mechanism is visible once stated — a model that is *consistently* wrong has a stable majority answer, so self-consistency is high exactly where accuracy is zero. Rounds B3 and B4 were therefore measuring the stability of a wrong answer, and B5 shows the same from the other side: in 11 of 19 problems all three original samples agree, and those answers are mostly wrong. *What a submission can buy is the stability of an error; what the label asks for is the fragility of a correct solution.* [78709ee: ttp_r4_report.txt]

Faithful replication is possible and still does not predict. Round D1 implemented the definition directly ($N{=}4$ samples, $K{=}6$ answer-preserving perturbations including model-written paraphrases that pass a number-multiset check, majority rule, truncated samples excluded, cap 24576). It reaches AUC 0.673 inside its null $[0.31, 0.70]$. The informative comparison is against a reduced version of itself: with $K{=}2$ and rename-only perturbations the AUC is 0.675. Raising diversity from two perturbation types to six moved the AUC by 0.002, so faithful replication is not the missing piece. [0fe94a2: h22_official33_skywork_report.txt]

A Condensed negative results

Three of these are structural rather than simple nulls, and they are the ones we would carry forward. (i) *The inference budget selects the method*: the generation-length family needs cap ≥ 8192, which a final-set run cannot buy, so the method that scores best offline is unavailable online. (ii) *The label definition is reproducible at test time but not predictable from behaviour*: H22 at AUC 0.673 inside its null, with perturbation diversity worth 0.002. (iii)

Table 3: All pre-registered hypotheses, one line each. "Primary" is the pre-registered primary metric on the organisers' 33 problem-level labels unless noted; nulls are 1000-permutation problem-level shuffles. "rev." marks a result opposite to the registered sign, which we did not flip. Logs are under `experiments/results/`.

	hypothesis (primary metric)	value	null 95%	verdict
H1	batch median split, 12-feature LOPO	—	—	no signal
H2	difficulty × capability (label consistency)	100%	—	labels are problem-level
H3	problem pooling, shared instrument	0.33	—	length only, rev.
H4	external validation, MATH→v2, layer 18	0.73	$[0.36, 0.75]$	no
H5	memorisation: NLL, Min-K%, zlib	0.69	—	length proxy
H6	internal shake under edits, d_{last}	0.29	below	rev.
H8	surface-gradient dependence	0.51	—	no
H9	essence separation (deletion importance)	0.596	$[0.32, 0.65]$	no
H10	trajectory compass, 4 models	0.615	$[0.28, 0.70]$	no
H11/H11b	surface-token knockout, all queries	0.403	—	no (effect ≈ 0)
H13	surface-subspace PCA energy	0.512	$[0.37, 0.64]$	no
H14/H14b	generation length ratio under rename	0.400	$[0.29, 0.70]$	rev.
H15	renameable surface symbol count	0.760	$[0.30, 0.70]$	✓ raw, no after length
H16/H16b	prefill pooled probe	0.531	$[0.31, 0.72]$	no
H18–H19	Min-K%++, prefix completion, NLL gap	0.368–0.490	—	no
H20	original-only generation length	0.850	$[0.30, 0.72]$	✓ but budget-bound
H20-M	per-model length, 3 models	0.823	—	saturation confound
H21a–d, H21p	self-uncertainty, forced-answer drop	0.802	$[0.28, 0.71]$	✓ then failed replication
H21cK	K constant-perturbation average	0.604	—	no (single-variant noise)
H22	faithful definition replication	0.673	$[0.31, 0.70]$	no
H23	membership inference, own-model labels	0.408	$[0.37, 0.63]$	no, length proxy
#77	final probe protocol, model mean	0.616	upper 0.78	no
#GEN-PROBE	mid-generation probe, window 512	0.619	upper 0.73	no
#CMP4	four classifiers	span 0.109	—	classifier is not the cause
#LIT-PLAN	entropy trajectory, logit lens, mass-mean	0.413–0.633	—	no

Answer-preserving perturbations cannot make the majority rule fire: per-type collapse 10.9% against the 40–50% the organisers' spurious rate implies.

B Resolution of our own evaluation

Table 4: AUC recovered by the real probe pipeline when a signal of known size is planted in one feature dimension, n=33 problems, PCA to 24 components, layer and C chosen inside folds.

feature dimension	1σ	3σ	8σ	20σ
10	0.669	0.996	1.000	1.000
40	0.715	0.873	0.927	0.946
400	0.515	0.585	0.685	**0.696**

Re-drawing rows is a second, independent read on the same ceiling. Holding the design, the labels and the evaluation path fixed and resampling rows to the same count gives 95% intervals of width 0.49–0.76 (200 draws per model) — wider than any design difference we measured. That is why §3 declines to rank its five classifiers. [`f730878`: `probe_subset_shift.txt`]

C Two measurement defects

A tokeniser-level one: vLLM detokenised one of the four models through a legacy path, leaving a byte-level marker for spaces, so the grader scored correct answers as wrong at a rate of 0.293. For two weeks this read as a model difference — lowest `base_acc`, highest spurious rate, below-chance AUC — and all of it was one bug, visible only

once we started saving raw generations. We re-generated the affected labels; about one third of them changed, and the model's mean original accuracy moved from 0.70 to 0.97. [abf4043: tok_audit]

An extraction-level one: the answer parser reads ANSWER: lines and inline \(\boxed{}\) but not display \[\boxed{}\], costing one model 5.0% of its measured accuracy while costing another 0%, because the choice of output format differs by model. [ff9cb8c: extract_fail_audit.txt]

Both defects inflate or deflate a model's apparent capability, both were invisible in the aggregates we had been storing, and both are the reason our standing rule is now that any generation run must save its raw text.

D Submissions

Methods were packaged as Codabench submissions; the following are the submission IDs visible on the public leaderboard for our account, and they are the IDs this report describes.

submission_id	track	accuracy	method
1000005	Main (N=21)	0.48	prior-calibrated length rule (§5)
1000047	Small (N=14)	0.50	prior-calibrated length rule
1005084	Small (N=14)	0.50	front-of-reasoning entropy, experimental

The natural-termination hybrid of §6 is the only method in Table 2 above both baselines, and it was packaged for both tracks after the measurements above (dist/hfin-main.zip, dist/hfin-small.zip; routing: Olmo and Skywork use the rule with a 12288 → 8192 per-case budget ladder, every other model_id including gpt-oss-120b at all effort levels takes the length rule without loading a model). At Main scale, four to five problems per model, the first rung costs 508–582 s per generating call against a ≈ 1650 s share, i.e. 31–35% of the budget. **It was never submitted.** Its only evidence is therefore the offline measurement on the 33 public labels, and we claim no leaderboard score for it.

Nominated submission. We nominate the mid-generation probe that Table 1 is about (dist/sub2048.zip), the one white-box method we did submit. Its observed scores were 0.62 on Main (13/21) and 0.57 on Small (8/14). Those runs predate the three IDs in the table and were overwritten on the per-track leaderboard, which shows only the latest submission per track, so the ID cannot be recovered from the public API. [TODO: the nominated submission_id, read from the Codabench "My Submissions" page]

Two measurements belong beside that nomination, and both are in the body rather than hidden here. First, its fixed threshold is miscalibrated off its training distribution: the predicted True rate is 0.48–0.50 on the labels it was fitted to and 0.15–0.21 on the official 33, where its accuracy falls below the constant predictor. Second, the same frozen weights transfer at pooled AUC 0.610, below the 0.671 of problem length alone. Removing either per-model partition — one instrument for every model, or one shared threshold — does not recover anything (rows P5a and P5b of Table 2); so whatever the Main score of 0.62 was, it was not the per-model routing. We nominate it because it is the submission this report analyses end to end, not because the number is defensible.

Reproducibility

Every number cites the commit that produced it; logs live in experiments/results/ and the pre-registration for each hypothesis, including the ones that failed, is in the versioned notebook file in the repository. Code, all measurement logs and every pre-registration are served anonymously at https://anonymous.4open.science/r/aimo-interp-log